Moveit Cartesian Path

Moveit Cartesian Path
- 1. Usage Environment
- 2. Driving the Real Machine
  - 2.1 Starting the Real Device
- 3. Starting MoveIt2
  - 3.1 Cartesian Path
  - 3.1. Visualization
  - 3.2. Launch Command
  - 3.3. Source Code Analysis

1. Usage Environment

ROS2: Humble

Enter the commands in the tutorial into the accompanying virtual machine to run the function.

2. Driving the Real Machine

Driving the real machine is achieved by subscribing to the Moveit2 `/joint_states` topic, converting the joint state information of the robotic arm into control data for the real robotic arm.

Note: Since the real robotic arm does not have obstacle avoidance capabilities, some poses may encounter obstacles; therefore, the planned robotic arm movements should be as reasonable as possible and avoid locations with obstacles.

(It is recommended to use preset poses for demonstrating the driving of the real machine.)

The motherboard and virtual machine need to be configured for multi-machine communication. The robotic arm and virtual machine need to be on the same network, and the virtual machine's `ros_domain_id` must be consistent with the `ros_domain_id` in Docker. (Both devices are pre-configured with the same `ros_domain_id` at the factory; only the virtual machine and the robotic arm need to be on the same network.)

2.1 Starting the Real Device

```
# Start this command in Docker,
ros2 run dofbot_ctrl SimulateToArm
```

3. Starting MoveIt2

```
# Start this command in the virtual machine,
ros2 launch dofbot_moveit demo.launch.py
```

```
yahboom@yahboom-VM:~$ ros2 launch dofbot_moveit demo.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-03-13-14-31-56-635718-yahboom-VM-9623
[INFO] [launch]: Default logging verbosity is set to INFO
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
Using load_yaml() directly is deprecated. Use xacro.load_yaml() instead.
[INFO] [robot_state_publisher-1]: process started with pid [9625]
[INFO] [move_group-2]: process started with pid [9627]
[INFO] [rviz2-3]: process started with pid [9629]
[INFO] [ros2_control_node-4]: process started with pid [9631]
[INFO] [spawner-5]: process started with pid [9633]
[INFO] [spawner-6]: process started with pid [9635]
[INFO] [spawner-7]: process started with pid [9637]
[robot_state_publisher-1] [WARN] [1773383518.815694424] [kdl_parser]: The root link base_link has an inertia specified in the URDF,
but KDL does not support a root link with an inertia.  As a workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-1] [INFO] [1773383518.815824354] [robot_state_publisher]: got segment Camera_Link
[robot_state_publisher-1] [INFO] [1773383518.815886691] [robot_state_publisher]: got segment Gripping_point_Link
[robot_state_publisher-1] [INFO] [1773383518.815895376] [robot_state_publisher]: got segment Llink1_Link
[robot_state_publisher-1] [INFO] [1773383518.815901628] [robot_state_publisher]: got segment Llink2_Link
[robot_state_publisher-1] [INFO] [1773383518.815907310] [robot_state_publisher]: got segment Llink3_Link
```

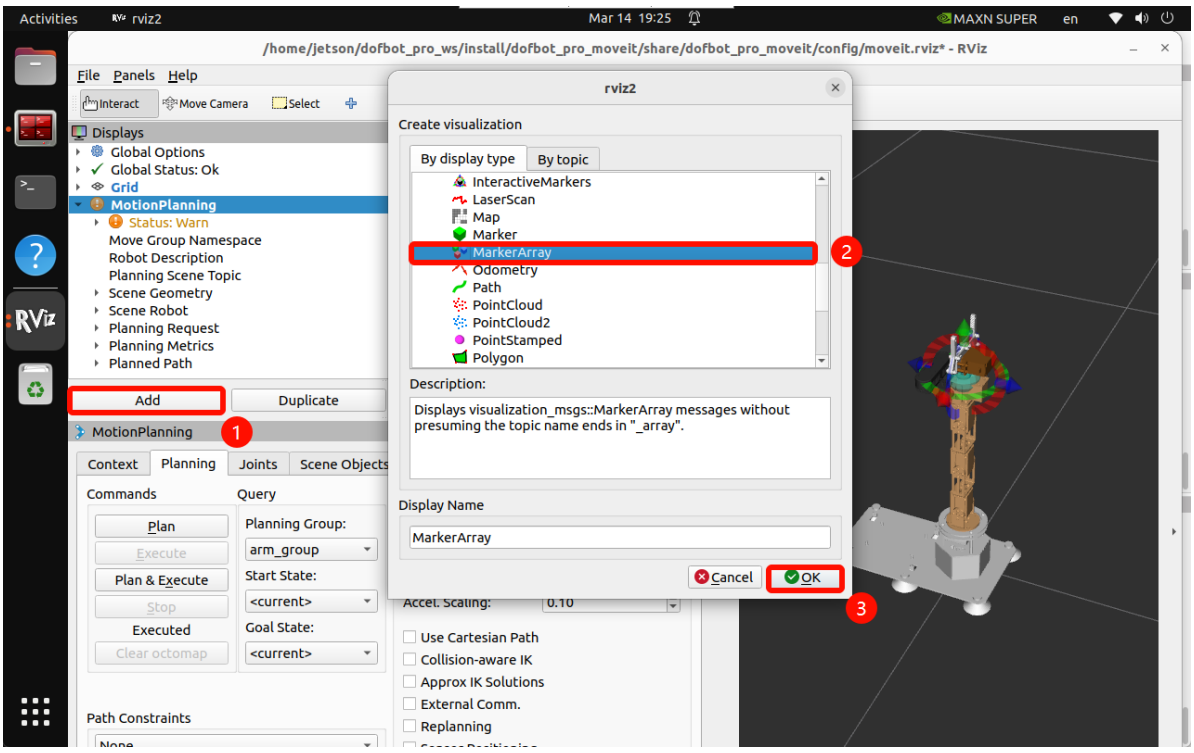
3.1 Cartesian Path

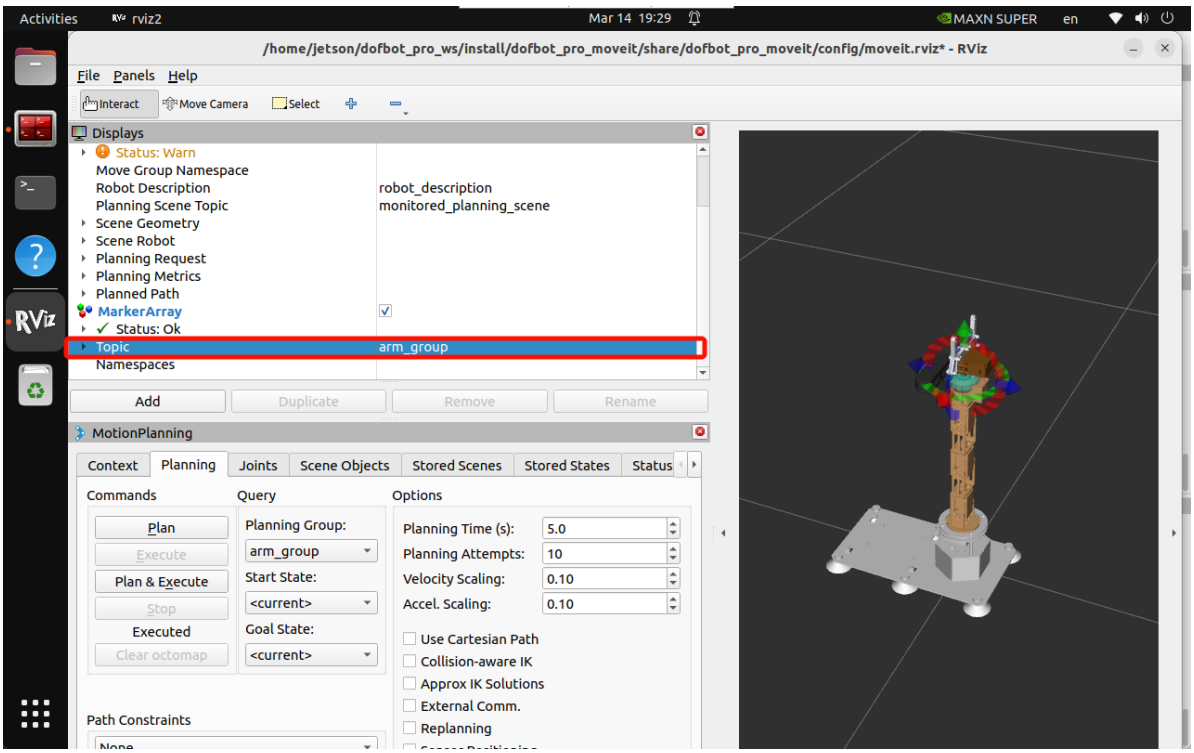
A Cartesian path refers to the linear motion path of the robotic arm's end effector in a Cartesian coordinate system.

```
Due to limitations regarding the robotic arm's degrees of freedom and structural configuration, identifying suitable Cartesian waypoints is extremely challenging.
```

3.1. Visualization

Before executing the launch command, you must add the `MarkerArray` plugin within `RViz2` to visualize the straight-line path generated by the Cartesian planner. For the `MarkerArray` plugin, you need to select the `arm_group` topic.

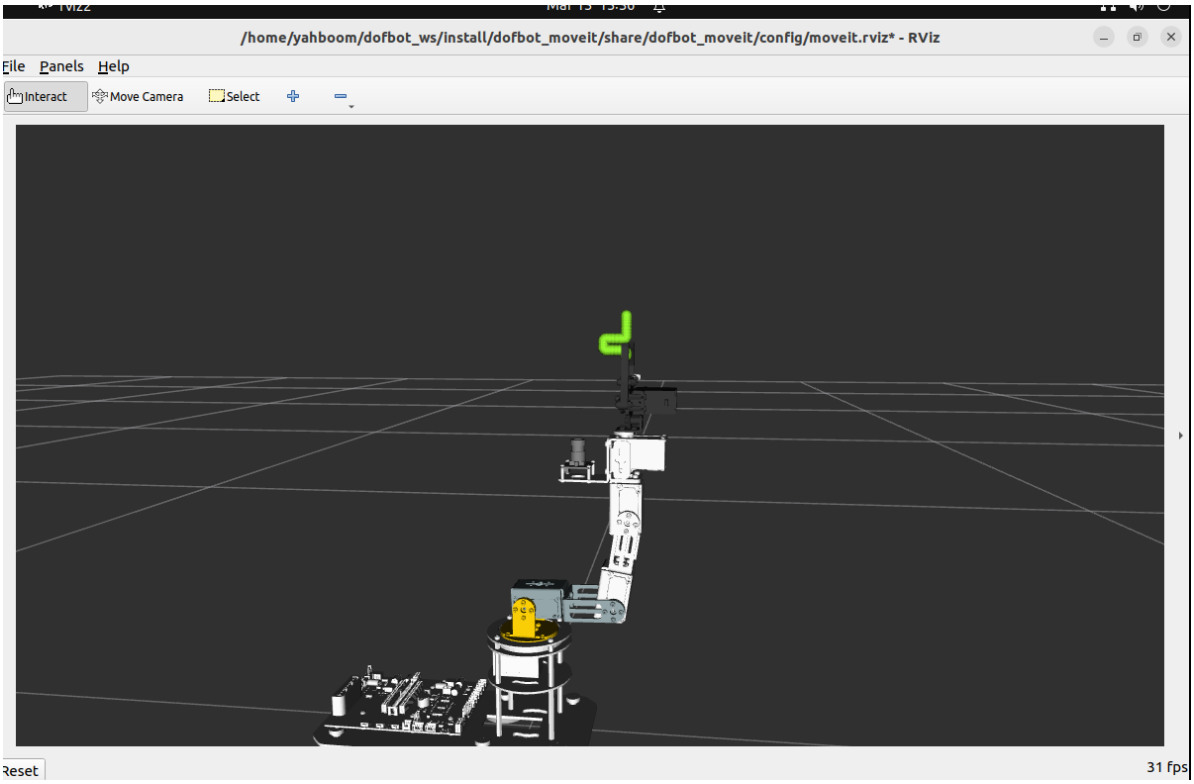




3.2. Launch Command

Ensure that the robotic arm has successfully loaded within MoveIt and that the message `You can start planning now!` appears. Then, execute the following command; the robotic arm will automatically plan its Cartesian path.

```
# Execute this command within the virtual machine
ros2 run dofbot_moveit cartesian_path
```



3.3. Source Code Analysis

Source Code Path:

```
#Path in the Virtual Machine
~/dofbot_ws/src/dofbot_moveit/src/cartesian_path.cpp
```

```
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <moveit_msgs/msg/display_trajectory.hpp>
#include <moveit_visual_tools/moveit_visual_tools.h>
#include <geometry_msgs/msg/pose.hpp>
#include <vector>
#include <moveit/robot_trajectory/robot_trajectory.h>
#include <moveit/robot_state/robot_state.h>
#include <moveit/robot_model/robot_model.h>
#include <moveit/robot_model_loader/robot_model_loader.h>
#include <moveit/robot_model/robot_model.h>
#include <moveit/robot_state/robot_state.h>

class CartesianPathPlanning : public rclcpp::Node
{
public:
  CartesianPathPlanning()
    : Node("cartesian_path_planning")
    {
      RCLCPP_INFO(this->get_logger(), "Initializing CartesianPathPlanning.");
    }

  void initialize()
  {
   // Load the robot model using RobotModelLoader
    robot_model_loader::RobotModelLoader robot_model_loader(shared_from_this(),
"robot_description");
    const moveit::core::RobotModelPtr& robot_model =
robot_model_loader.getModel();

   // Initialize MoveGroupInterface
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>
(shared_from_this(), "arm_group");
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    const moveit::core::JointModelGroup* joint_model_group = robot_model-
>getJointModelGroup("arm_group");

    moveit_visual_tools::MoveItVisualTools visual_tools(shared_from_this(),
"base_link", "arm_group", move_group_interface_->getRobotModel());



    // Define the initial position point
    geometry_msgs::msg::Pose initial_pose;
    initial_pose.position.x = 0.093;
    initial_pose.position.y = -0.003;
```

```
    initial_pose.position.z = 0.324;
    initial_pose.orientation.x = 0.0;
    initial_pose.orientation.y = 0.0;
    initial_pose.orientation.z = 0.0;
    initial_pose.orientation.w = 1.0;


    // Move the robotic arm to the initial position.
    move_group_interface_->setPoseTarget(initial_pose);
    bool success = (move_group_interface_->move() ==
moveit::core::MoveItErrorCode::SUCCESS);

    if (success)
    {
      RCLCPP_INFO(this->get_logger(), "Moved to initial position
successfully.");

      // Define a series of target poses
      std::vector<geometry_msgs::msg::Pose> waypoints;
      geometry_msgs::msg::Pose target_pose = initial_pose;

      // Add some waypoints
      target_pose.position.z -= 0.02;
      waypoints.push_back(target_pose);
      target_pose.position.x -= 0.02;
      waypoints.push_back(target_pose);
      target_pose.position.z -= 0.01;
      waypoints.push_back(target_pose);
      target_pose.position.x += 0.02;
      waypoints.push_back(target_pose);
      target_pose.position.z -= 0.02;
      waypoints.push_back(target_pose);
      target_pose.position.x -= 0.02;
      waypoints.push_back(target_pose);
     // Print all waypoints
      RCLCPP_INFO(this->get_logger(), "Waypoints:");
      for (size_t i = 0; i < waypoints.size(); ++i)
      {
        RCLCPP_INFO(this->get_logger(), "Waypoint %zu: [x: %f, y: %f, z: %f]",
i, waypoints[i].position.x, waypoints[i].position.y, waypoints[i].position.z);
      }

      // Calculate Cartesian Path
      moveit_msgs::msg::RobotTrajectory trajectory_msg;
      double jump_threshold = 0.0;
      double eef_step = 0.005;
      double fraction = move_group_interface_->computeCartesianPath(waypoints,
eef_step, jump_threshold, trajectory_msg);

      if (fraction >= 0.0)
      {
        RCLCPP_INFO(this->get_logger(), "Cartesian path planned successfully
(%.2f%% achieved)", fraction * 100.0);

       // Convert moveit_msgs::msg::RobotTrajectory to
robot_trajectory::RobotTrajectory
        robot_trajectory::RobotTrajectory trajectory(move_group_interface_-
>getRobotModel(), move_group_interface_->getName());
```

```
        // Visualize the trajectory in RViz
        visual_tools.publishTrajectoryLine(trajectory_msg,
move_group_interface_->getRobotModel()->getLinkModel("Gripping_point_Link"),
joint_model_group, rviz_visual_tools::LIME_GREEN);
        visual_tools.trigger();

        // Execute the planned path
        moveit::planning_interface::MoveGroupInterface::Plan plan;
        plan.trajectory_ = trajectory_msg;
        move_group_interface_->execute(plan);
      }
      else
      {
        RCLCPP_ERROR(this->get_logger(), "Failed to compute Cartesian path");
      }
    }
    else
    {
      RCLCPP_ERROR(this->get_logger(), "Failed to move to initial position");
    }
  }

private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
  std::shared_ptr<moveit_visual_tools::MoveItVisualTools> visual_tools_;
};

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<CartesianPathPlanning>();

  rclcpp::executors::SingleThreadedExecutor executor;
  executor.add_node(node);
  // Start asynchronous spinner
  node->initialize();
  executor.spin();

  rclcpp::shutdown();
  return 0;
}
```